

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Análisis y Diseño I.
Ing. Daniel Gonzales.



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

FITNESS APP - MANUAL TÉCNICO

Estudiante	Carné
201731523	Byron Fernando Torres Ajxup
201931811	Diego José Maldonado Monterroso
202031398	Miguel Angel Quemé Gómez
202231207	Kevin Alexander Ramírez Mushín

Quetzaltenango, Guatemala.

Lunes 17 de agosto de 2026.

DESCRIPCIÓN

Este manual describe cómo está construido, cómo se levanta y cómo se opera el sistema completo: la API REST y el SPA que la consume. El contrato de la API, endpoint por endpoint, no se transcribe aquí: lo genera SpringDoc a partir del propio código y se consulta en `/swagger-ui.html` o en crudo en `/v3/api-docs`. Al salir del código, siempre está sincronizado con lo que corre.

	Backend	Frontend
Repositorio	Backend/Fitness-App	Frontend/fitness-app
Stack	Java 25 . Spring Boot 4.1.0 . PostgreSQL 17	React 19.2 . TypeScript 6 . Vite 8 . MUI 5
Puerto	8080	5173
Imagen	djmaldonado19/fitness-app-backend: latest	djmaldonado19/fitness-app-front end: latest
Tamaño	293 archivos Java . 32 controladores . 129 endpoints . 109 DTOs	152 archivos . 13 modulos . 49 pantallas . 52 rutas
Base de datos	31 tablas . 97 restricciones CHECK con nombre ck_	
Catálogo de errores	80 códigos en ErrorCode	getErrorMessage() los muestra tal cual
Pruebas	89 unitarias en 14 clases . Postman con 461 aserciones	tsc -b + ESLint en CI

Los dos se despliegan en la misma instancia EC2 (**18.227.211.214**) como dos stacks de compose independientes.

ARQUITECTURA DEL SISTEMA

1.1 Backend — monolito modular.

Monolito modular sobre Spring Boot. Un paquete por dominio, y dentro de cada uno la misma estructura: controller (HTTP), service (reglas de negocio), repository (persistencia), model (entidades JPA) y dto (contrato de entrada y salida). Ninguna entidad JPA sale a la red: todo lo que cruza el borde HTTP es un record de dto.

com.fitness.app

— iam	autenticación, JWT, doble factor, cuentas de usuario
— directory	personas, socios, empleados, entrenadores
— membership	planes, contratación, congelamiento, renovación
— access	control de acceso, visitas, pases de invitado
— classes	clases grupales, sesiones, inscripción, lista de espera
— training	asignación de entrenador, rutinas, mediciones, notas, alertas
— nutrition	catálogo de alimentos, comidas, meta calórica, resumen
— billing	promociones y pagos
— report	los 9 reportes y su exportación
— notification	avisos persistidos para el socio
— common	ErrorCode, BusinessException, GlobalExceptionHandler, ErrorResponse
— config	seguridad, JWT, CORS, OpenAPI, propiedades del gimnasio

1.2 Frontend — módulos por feature.

El SPA aplica la misma idea que el backend, con otro vocabulario: un módulo por dominio, y dentro de cada uno siempre el mismo contrato interno. Lo transversal vive fuera de src/modules/.

src/

— api/client.ts	axios: baseUrl, interceptor de Bearer, manejo de 401
— auth/	AuthContext, useAuth, ProtectedRoute, permissions
— layouts/	AppLayout (barra + Sidebar), PublicLayout
— components/	AppDatePicker, SimpleBarChart, TrendLineChart
— router.tsx	las 52 rutas y sus guardas por rol
— modules/	13 módulos: auth, dashboard, members, employees, trainers, membership, billing, access, classes, training, nutrition, reports, notifications

Cada módulo, sin excepción:

services.ts	una funcion por endpoint. Solo HTTP, cero logica
hooks.ts	useQuery / useMutation sobre services.ts. Aqui viven los snackbars y las invalidaciones de cache
types.ts	copia literal de los DTOs del backend, en snake_case
pages/	una pantalla por ruta
components/	componentes propios del módulo

1.3 Recorrido de una petición.

Navegador (:5173)

| fetch con Authorization: Bearer <jwt>
| cruza origen: :5173 → :8080 (no hay proxy)



CorsConfigurationSource — ¿el Origin está en CORS_ALLOWED_ORIGINS? — no → el navegador bloquea

| sí



JwtAuthenticationFilter — firma y vigencia del token — inválido → 401 UNAUTHENTICATED

| construye AuthenticatedUser(appUserId, username, role)



SecurityConfig — ¿el rol alcanza para esta ruta y método? — no → 403 FORBIDDEN_RESOURCE

| sí



Controller — @Valid sobre el cuerpo — inválido → 400 VALIDATION_ERROR + field_errors



Service — reglas de negocio + autorización por fila → TRAINER_SCOPE_VIOLATION / FORBIDDEN_RESOURCE

| (memberService.findById(id, principal))



Repository → PostgreSQL — CHECK o índice único → 409 CONSTRAINT_VIOLATION

1.4 Reportes y exportación.

Los 9 reportes desembocan en un único punto, ReportExporter, que decide el formato. Un ReportTable aplanar cualquier DTO de reporte en cabeceras y celdas usando el mismo ObjectMapper de la aplicación, y cuatro renderizadores lo escriben. Añadir un formato es una constante del enum y una rama; no cuesta nada por reporte.

Formato	Cómo se produce	Content-Type
JSON	serialización normal	application/json
CSV	ReportCsv, RFC 4180	text/csv; charset=UTF-8
XLSX	ReportXlsx, Apache POI. Los importes se escriben como celdas numericas, no como texto, para poder sumarlos	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
PDF	ReportPdf, OpenPDF, A4 apaisado	application/pdf
PNG	ReportPng, ImageIO y Graphics2D del JDK, sin dependencia	image/png

STACK TECNOLÓGICO

2.1 Backend.

Componente	Versión	Para qué
Java	25	Lenguaje. record para los 109 DTOs
Spring Boot	4.1.0	Parent POM; gestiona lo que no está fijado abajo
Spring Web MVC	gestionada	Los 32 controladores sobre Tomcat embebido
Spring Data JPA + Hibernate	gestionada	Persistencia, con dd1-auto: validate
Spring Security	gestionada	Cadena de filtros, BCrypt, reglas por ruta y rol
JWT	0.13.0	Token de sesion HS256
Spring Validation	gestionada	Bean Validation sobre los cuerpos
Spring Mail	gestionada	Códigos 2FA y de recuperación
SpringDoc OpenAPI	3.1.0	/swagger-ui.html y /v3/api-docs
Apache POI	5.4.1	Exportación XLSX
OpenPDF	2.0.3	Exportación PDF
PostgreSQL	17 (postgres : 17-alpine)	Base de datos
Lombok	gestionada	Boilerplate de entidades y servicios. Excluido del jar
Maven	3.9.16 (wrapper)	Construcción, sobre maven:3.9-eclipse-temurin-25
JUnit 5 + Mockito	gestionada	Las 89 pruebas unitarias

2.2 Frontend.

Componente	Versión	Para qué
React	19.2.8	Interfaz, con React Compiler vía Babel
TypeScript	6.0.3	Tipado; types. ts copia los DTOs del backend
Vite	8.2.1	Servidor de desarrollo y build
MUI (Material UI)	5.18.0	Componentes, con @mui/lab para LoadingButton
Emotion	11.14	Motor de estilos de MUI
React Router	7.18.2	Las 52 rutas y las guardas por rol
TanStack Query	5.101.4	Cache e invalidacion del estado del servidor
Axios	1.19.0	Cliente HTTP con interceptores de token y de 401
React Hook Form + Yup	7.85.0/ 1.7.1	Formularios y validación
AG Grid	36.1.0	Tablas de los listados grandes
MUI X Date Pickers + dayjs	7.29.4 / 1.11.21	Fechas, con locales
notistack	3.0.2	Snackbars de éxito y error
ESLint + typescript-eslint	10.8.1 / 8.67	Linter, obligatorio en CI
Node.js	24 en CI (mínimo 20.19)	Entorno de ejecución

MODELO DE DATOS

Dominio	Tablas
Identidad	person, app_user, verification_code
Directorio	member, employee, trainer, trainer_specialty
Membresías	membership_plan, membership, membership_freeze
Acceso	facility_visit, guest_pass
Facturación	promotion, payment
Clases	group_class, class_session, class_enrollment, waitlist_entry, class_rating
Entrenamiento	trainer_assignment, exercise, routine, routine_exercise, progress_measurement, trainer_note, trainer_alert
Nutrición	food, meal, meal_item, nutrition_goal
Notificaciones	notification

Reglas que viven en la base, no en el código, porque son invariantes que ninguna ruta debe poder violar: 97 CHECK con nombre (ck_membership_dates, ck_promotion_pct, ck_food_macros...), la restricción uq_group_class_slot (un entrenador no puede tener dos clases el mismo día a la misma hora) y el índice único parcial uq_membership_in_force, que impide dos contratos vigentes por socio precisamente porque solo aplica a las filas ACTIVE o FROZEN.

Cuando una de esas reglas salta, GlobalExceptionHandler la traduce a 409 CONSTRAINT_VIOLATION y escribe el nombre de la restricción en el log, nunca en la respuesta.

Nota de operación: no hay Flyway ni Liquibase. schema.sql y data.sql los ejecuta el contenedor de Postgres solo cuando el volumen está vacío. Cambiar el esquema obliga a recrear el volumen postgres_data, y con ddl-auto: validate un despliegue contra un esquema viejo falla al arrancar en vez de corromper datos.

SEGURIDAD

Autenticación por JWT. POST /auth/login devuelve un token de 60 minutos (app.security.jwt.access-minutes). JwtAuthenticationFilter lo valida en cada petición y SecurityConfig decide el acceso por ruta.

Cuatro roles: ADMIN, RECEPTIONIST, TRAINER, MEMBER. Las reglas se declaran una sola vez, en SecurityConfig; por ejemplo /api/v1/reports/** es exclusivo de ADMIN.

Doble factor. Si la cuenta lo tiene activo, el login responde 202 con un challenge_id y un destino enmascarado (a***n@fitnessapp.local) en lugar del token; el token se emite al verificar el código en POST /auth/challenges/{id}/verifications.

- Código de 6 dígitos generado con SecureRandom.
- Se guarda con BCrypt, nunca en claro.
- Vence a los 10 minutos y admite 3 intentos; agotados, el código pasa a EXPIRED y la respuesta sugiere RESEND_CODE.

Autorización por fila. Que el rol permita la ruta no basta: un entrenador solo ve a los socios que tiene asignados y un socio solo su propio expediente. Los servicios lo comprueban al leer, y devuelven TRAINER_SCOPE_VIOLATION o FORBIDDEN_RESOURCE.

CORS. El SPA cruza origen en cada llamada (:5173 → :8080), así que la política vive en un único CorsConfigurationSource de SecurityConfig:

Ajuste	Valor	Por qué
Orígenes	CORS_ALLOWED_ORIGINS, lista separada por comas	Nunca *: con allowCredentials(true) el navegador lo rechaza
Métodos	GET POST PUT PATCH DELETE OPTIONS	OPTIONS porque cada petición con Authorization dispara preflight
Cabeceras expuestas	Content-Disposition	No está en la lista segura de CORS; sin exponerla el SPA no puede leer el nombre de archivo de un reporte exportado

MANEJO DE ERRORES

Toda respuesta de error tiene la misma forma, con los campos nulos omitidos:

```
{
  "error_code": "MEMBERSHIP_EXPIRED",
  "message": "La membresía venció.",
  "suggested_action": "RENEW_MEMBERSHIP",
  "timestamp": "2026-08-13T06:18:44.238Z",
  "path": "/api/v1/trainer-assignments",
  "field_errors": { "start_date": "must be a date in the present or in the future" },
  "trace_id": "406606d6-..."
}
```

ErrorCode es el catálogo completo: cada constante lleva su estado HTTP, su suggested_action y su mensaje, y ese es el único sitio donde se decide el código de una falla. trace_id aparece solo en el 500, que es el único caso en el que hace falta cruzar la respuesta con el log.

Situación	Handler	Respuesta
Regla de negocio	handleBusiness	la del ErrorCode
Anotacion de validacion	handleValidation	+ field_errors 400
JSON ilegible	handleUnreadableBody	400
Parametro invalido o ausente	handleInvalidParameter	400
Ruta inexistente	handleUnknownRoute	404 ROUTE NOT FOUND
CHECK o indice único	handleDataIntegrity	409 CONSTRAINT_VIOLATION
Cualquier otra cosa	handleUnexpected	500 + trace_id

JOBS

Dos tareas @Scheduled en MembershipService. Ningún endpoint las dispara: se demuestran con sus pruebas unitarias.

Cron	Método	Qué hace
050 * * * (00:05)	expireMemberships()	Marca EXPIRED los contratos que llegaron a su fecha de fin, cancela sus inscripciones futuras y notifica al socio
0 0 7 * * * (07:00)	notifyExpiringMemberships()	Avisa a quienes vencen dentro de gym.membership.expiry-notice-days (5 por defecto)

Como la columna status solo se pone al día a las 00:05, el reporte de membresías deriva el estado de end_date en lugar de leer la columna. Los servicios de negocio también verifican por fecha, de modo que el control de acceso es correcto aunque la columna vaya con retraso.

POLÍTICAS PARAMETRIZABLES

Las reglas que el enunciado deja al equipo están en `application.yml`, bajo `gym:`, y no repartidas por el código:

`gym:`

```
freeze: { max-days-per-cycle: 15, max-count-per-cycle: 2, cycle-days: 90 }
guest-pass: { max-free-per-person: 1 }
classes: { cancellation-window-hours: 2, waitlist-confirmation-minutes: 60 }
membership: { expiry-notice-days: 5 }
nutrition: { default-tolerance-percent: 10 }
```

La zona horaria no está aquí, y es a propósito. `compose.yml` fija TZ: America/Guatemala en los dos servicios. Las 32 llamadas a `LocalDate.now()` resuelven en la zona por defecto de la JVM, y los `DEFAULT CURRENT_DATE` del esquema (`member.joined_on`, `trainer_assignment.start_date`) en la del servidor de base de datos: una sola variable alinea las dos. En UTC —el valor por defecto de las imágenes— las seis horas posteriores a las 18:00 locales ya cuentan como el día siguiente, y un socio que registraba una comida a las 19:00 no podía editarla un minuto después porque `MealService.assertSameDay` la comparaba contra una fecha ya rodada.

Va escrita y no como `${TZ:-...}`: con la forma parametrizada, un servidor que exporte TZ en su entorno ganaría la sustitución de `compose` y desharía la corrección en silencio. Postgres la lee en el `initdb`, así que un volumen ya creado conserva su `postgresql.conf`: hay que sembrar para que la tome.

DESARROLLO LOCAL

Backend.

```
cd Backend/Fitness-App
docker compose up -d --wait      # postgres + app, API en :8080
```

Levanta `fitness_postgres` (`postgres:17-alpine`, volumen `postgres_data`) y `fitness_backend`. Postgres ejecuta `db/schema.sql` y `db/data.sql` solo cuando el volumen se crea vacío.

Para desarrollar con depurador, la aplicación puede correr en el host contra la base en contenedor:

```
docker compose up -d postgres --wait
mvn spring-boot:run
```

`application.yml` importa `./env` si existe, así que ambas formas toman las mismas credenciales sin exportar nada. `JWT_SECRET` debe tener al menos 32 bytes.

Para rehacer el esquema tras editar `schema.sql`, borra solo el volumen de datos — `down -v` también se llevaría `maven_repo`:

```
docker compose down
docker volume rm fitness-app-backend_postgres_data
docker compose up -d --wait
```

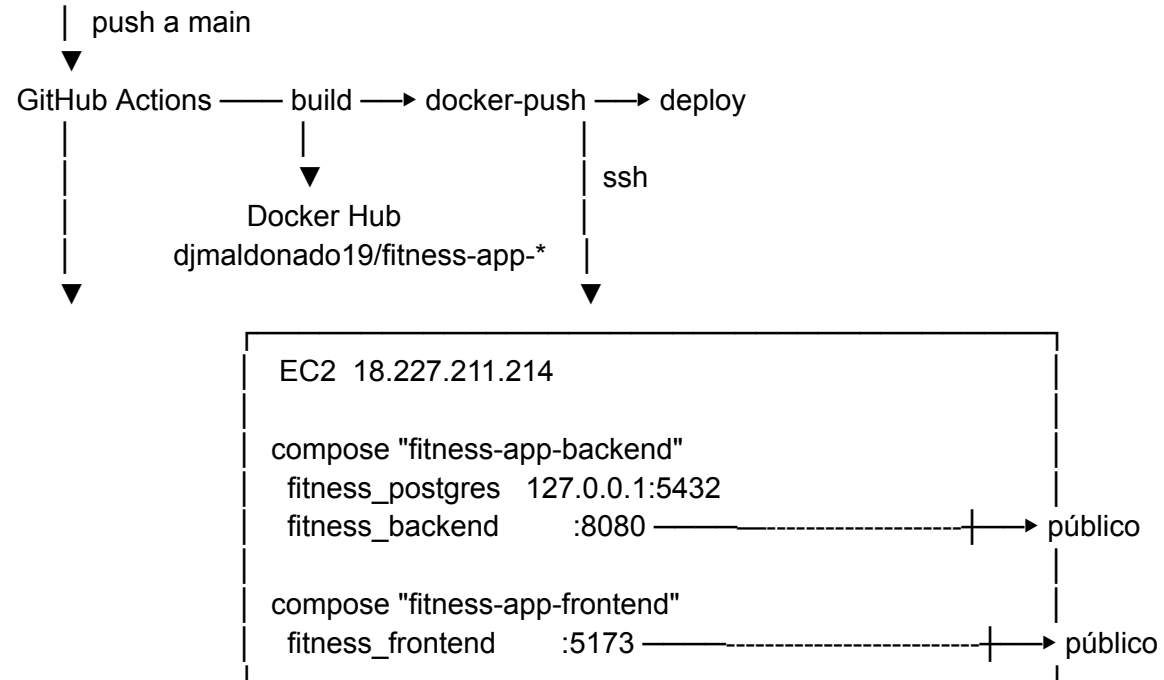
Frontend.

```
cd Frontend/fitness-app
npm ci
cp .env.example .env.local
npm run dev      # SPA en :5173
```

Se puede acceder con `admin / Admin123*`, el administrador que siembra `data.sql`.

INFRAESTRUCTURA

GitHub (2 repos): [ayd-practica1-frontend](#) y [ayd-practica1-backend](#)

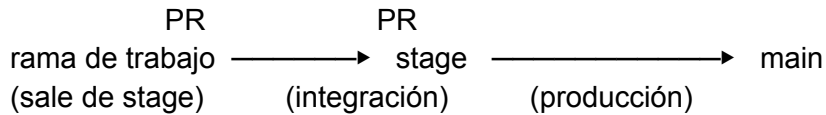


Pieza	Qué aporta
GitHub Actions	Integración continua en cada push y entrega continua solo desde main
Docker Hub	Registro de las dos imagenes, etiquetadas <run_number>, sha -< corto> y latest
EC2	Una sola instancia con dos stacks de compose independientes, uno por repositorio
Volúmenes	postgres_data (datos), maven_repo (cache de dependencias), build_output, node_modules

Los dos stacks no comparten red de Docker. Se hablan por el puerto publicado del host, igual que lo haría el navegador. Postgres es la excepción: se publica en 127.0.0.1:5432, así que no es accesible desde fuera de la instancia.

GITFLOW

Tres niveles de rama, y ningún commit directo sobre las dos protegidas. Es idéntico en los dos repositorios.



Rama	Qué es Cómo entra el código
main	Lo que esta desplegado en la EC2 Solo por PR desde stage
stage	Integracion: aquí se juntan los modulos antes de salir Solo por PR desde una rama de trabajo
rama de trabajo	Una funcionalidad o correccion Se crea desde stage, nunca desde main

```
git checkout stage && git pull origin stage
git checkout -b dmaldonado/<tema>
# ...trabajo...
git push -u origin dmaldonado/<tema> # PR contra stage
```

Nombres reales del historial: en el backend dmaldonado/<tema> (dmaldonado/iam-module, dmaldonado/reports-module, dmaldonado/final-audit); en el frontend conviven feature/<tema> (feature/billing, feature/classes) y dmaldonado/<tema>.

Por qué dos niveles y no uno. El despliegue se dispara con el merge a main. Si las ramas de trabajo apuntan ahí directamente, cada funcionalidad a medio integrar saldría a producción. stage es donde los módulos se encuentran por primera vez, que es donde aparecen los desajustes de contrato entre dos personas trabajando en paralelo.

Rama	build	docker-push	deploy
rama de trabajo	sí	no	no
stage	sí	no	no
main	sí	sí	sí

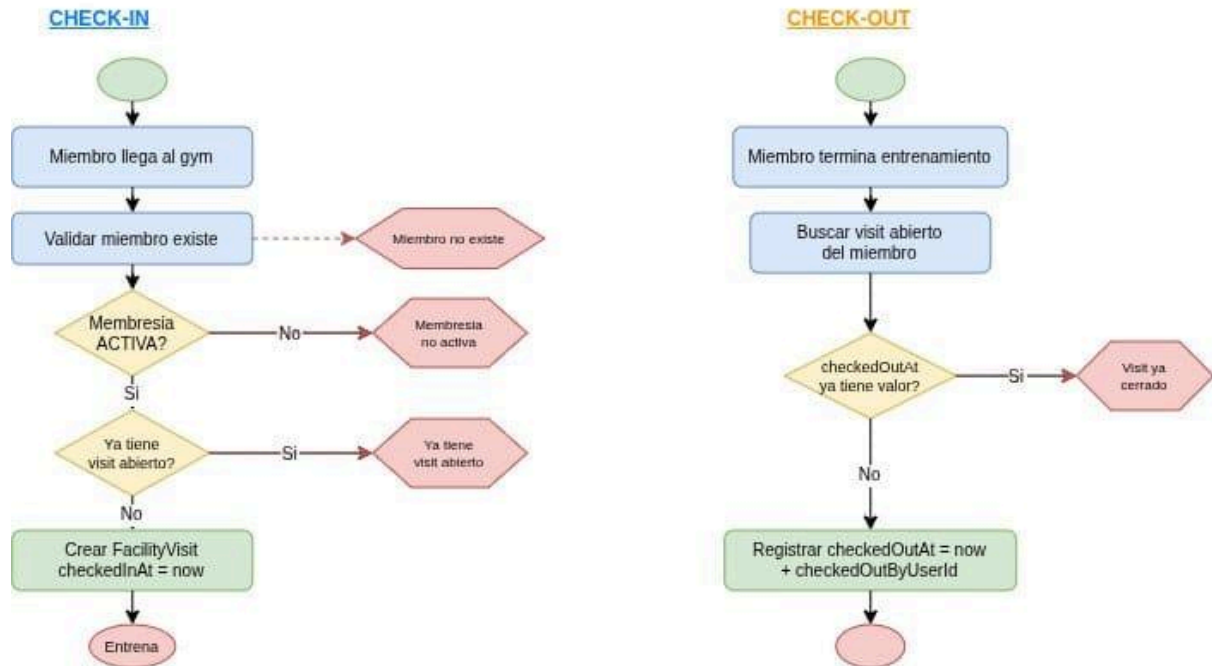
PRUEBAS

Qué	Cómo se corre	Estado
89 pruebas unitarias, 14 clases	mvn test	89/89, BUILD SUCCESS
Coleccion de Postman, 461 aserciones	node scripts/validate-api.js	461/461

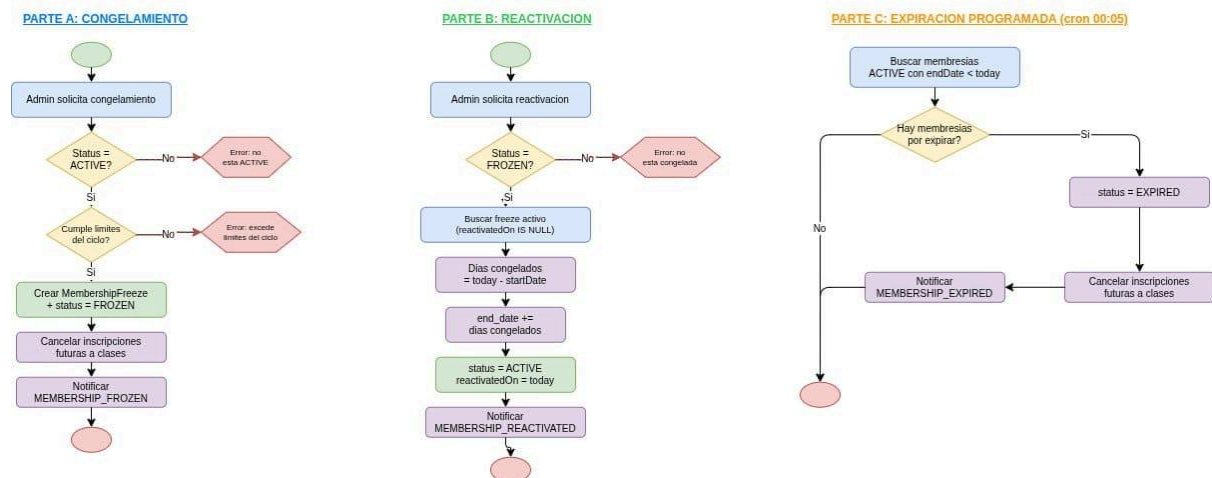
Las unitarias son JUnit 5 con Mockito sin contexto de Spring: se ejecutan en segundos y prueban reglas de negocio, no cableado. La colección (Docs/Fitness-App.postman_collection.json) recorre la API contra un backend vivo; el runner añade en memoria un script que rellena los códigos de doble factor leyéndolos del log, porque son datos que solo conoce el usuario.

ANEXOS

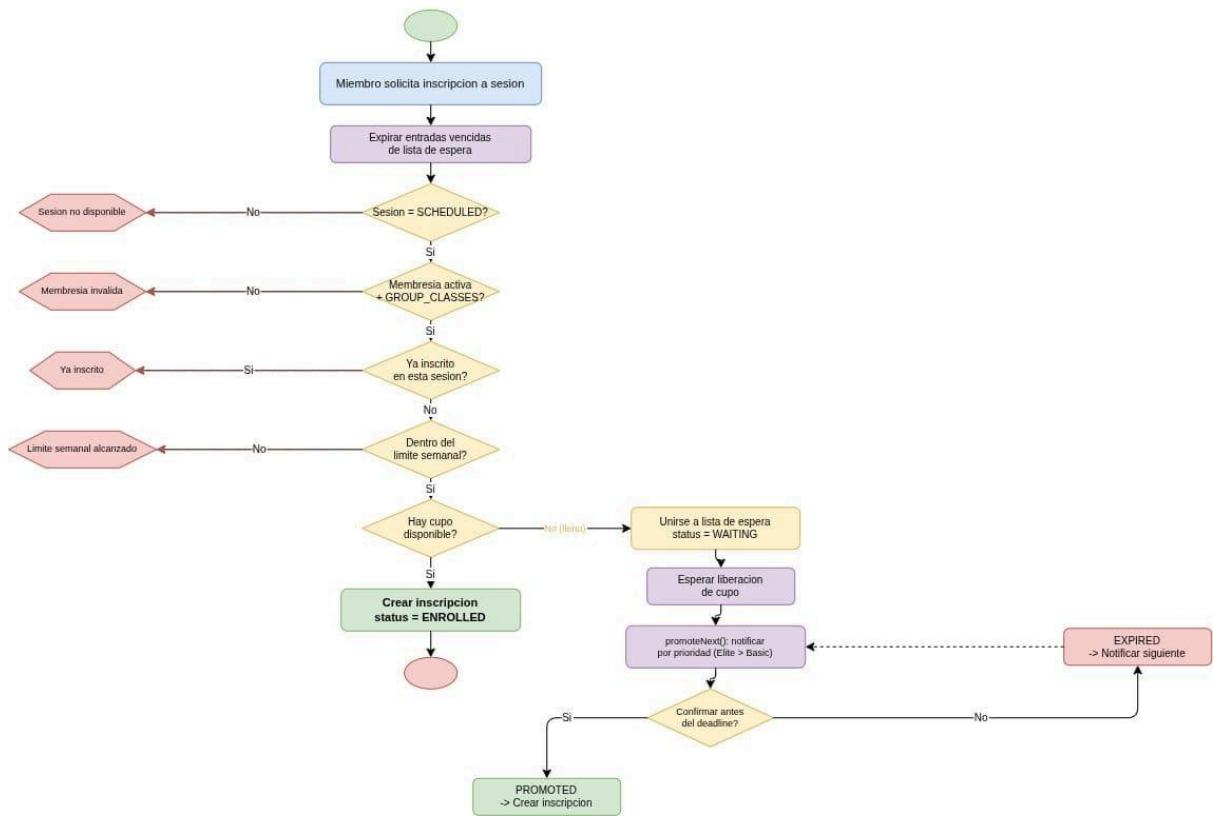
CHECK-IN / CHECK-OUT DE INSTALACION



CONGELAMIENTO Y REACTIVACION DE MEMBRESIA

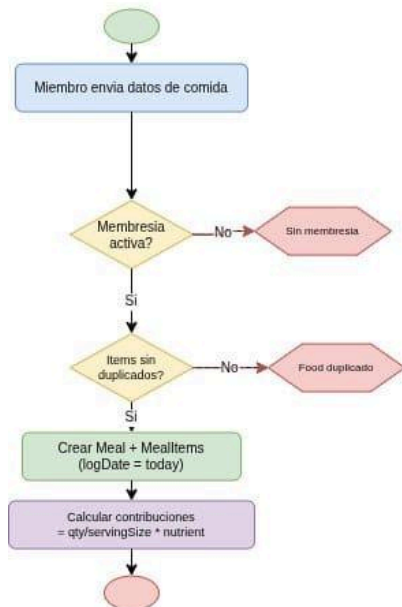


INSCRIPCION A CLASE GRUPAL

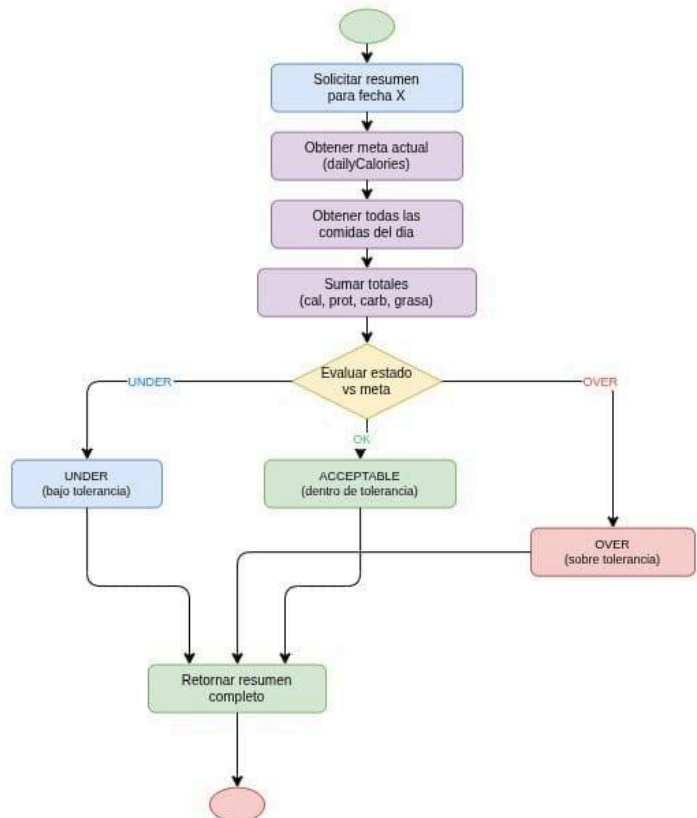


REGISTRO DE COMIDA Y RESUMEN NUTRICIONAL

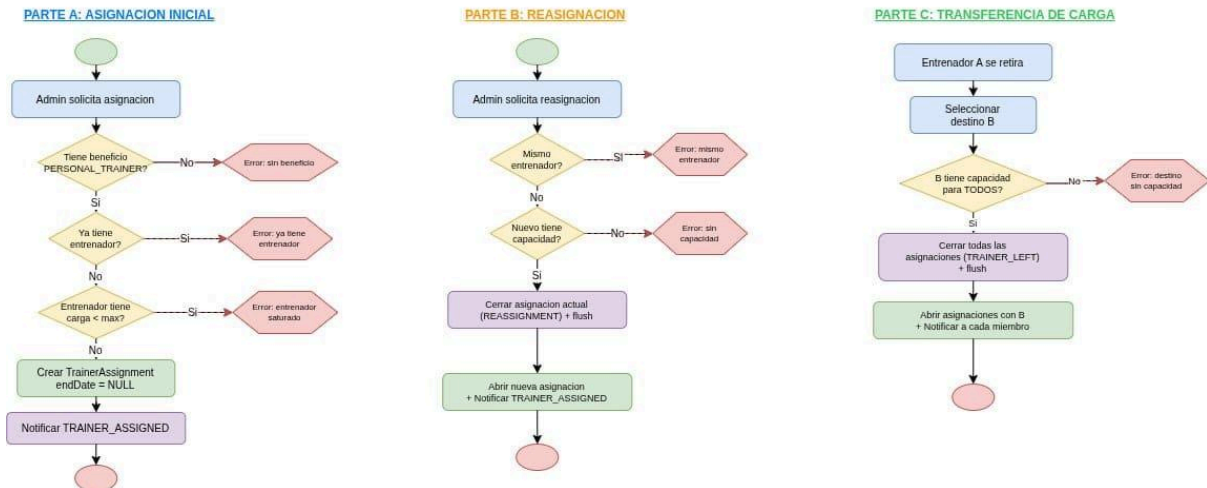
PARTE A: REGISTRAR COMIDA



PARTE B: RESUMEN NUTRICIONAL DIARIO



ASIGNACION Y REASIGNACION DE ENTRENADOR



REGISTRO DE PAGO

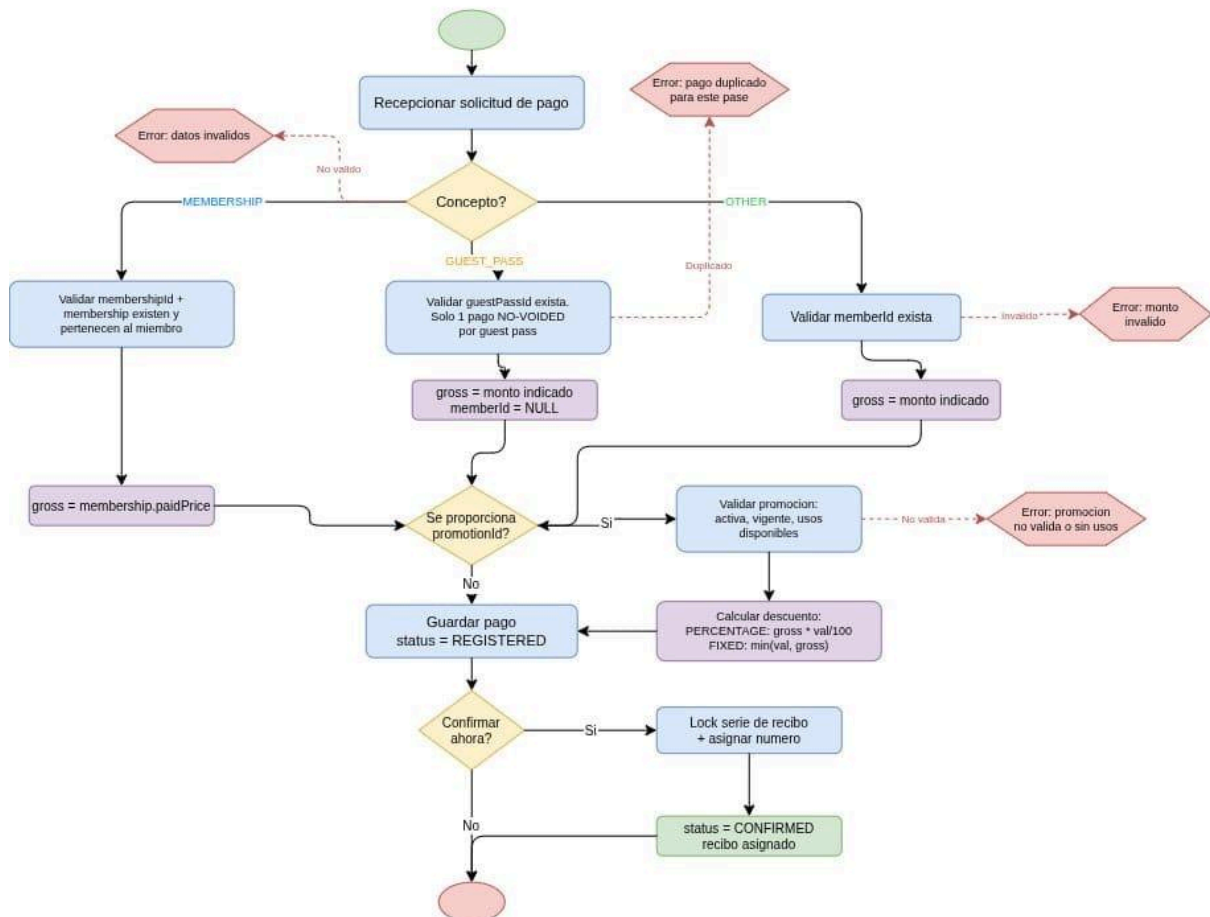


Diagrama Entidad Relación: [ER - Fitness App](#)

Se subió en formato PDF por el tamaño, ya que no se logra distinguir en formato PNG.